

Module 7 Advance Python Programming

1. Printing on Screen :-

Introduction to the print() function in Python :-

The **print()** function in Python is used to **display output on the screen**. It is one of the most basic and commonly used functions in Python programming. Programmers use it to show messages, values of variables, results of calculations, and other information to the user.

The print() function can display **text, numbers, variables, and expressions**. It helps in testing programs and understanding how the program works.

Syntax :- print(object)

• Formatting outputs using f-strings and format() :-

In Python, **output formatting** is used to display variables and values inside a string in a clear and readable way. Two common methods for formatting output are **f-strings** and the **format() method**.

1. Using f-strings

f-strings (formatted string literals) were introduced in Python 3.6. They allow variables and expressions to be directly included inside a string by placing them inside **curly braces {}** and adding **f before the string**.

2. Using format() Method

The **format() method** is another way to insert values into a string. In this method, **placeholders {}** are used in the string, and the values are passed inside the format() function.

2. Reading Data from Keyboard :-

• Using the input() function to read user input from the keyboard :-

The **input()** function in Python is used to **take input from the user through the keyboard**. It allows the program to receive data from the user while the program is running.

When the input() function is executed, the program **waits for the user to type something and press Enter**. The entered value is then stored in a variable.

Syntax :- input("message")

- **Converting user input into different data types (e.g., int, float, etc.).**

In Python, the `input()` function always returns the data as a string. If the user enters numbers and we want to perform calculations, we must convert the input into the required data type such as int, float, etc. This process is called type conversion.

1. Converting Input to Integer (int)

The `int()` function is used to convert user input into an integer.

2. Converting Input to Float (float)

The **`float()` function** converts the input into a decimal number.

3. Converting Input to String (str)

Sometimes we convert values to **string format** using the **`str()` function**.

3. Opening and Closing Files:-

- **Opening files in different modes ('r', 'w', 'a', 'r+', 'w+')**

In Python, files are opened using the **`open()` function**. While opening a file, we must specify a **mode** that tells Python how the file will be used (reading, writing, or appending).

Syntax

```
file = open("filename", "mode")
```

1. Read Mode ('r')

- Used to read the contents of a file.
- The file **must already exist**, otherwise Python will give an error.

2. Write Mode ('w')

- Used to **write data into a file**.
- If the file **does not exist**, it will be created.
- If the file **already exists**, its previous content will be **deleted**.

3. Append Mode ('a')

- Used to **add new data at the end of a file**.
- Existing data **is not removed**.

4. Read and Write Mode ('r+')

- Used to **read and write in the same file**.
- The file **must exist**.

5. Write and Read Mode ('w+')

- Used to **write and read the file**.
- If the file exists, **old data is deleted**.

- **Using the open() function to create and access files.**

In Python, the **open() function** is used to **create, open, and access files**. It allows a program to read data from a file or write data into a file.

The open() function requires the **file name** and the **mode** in which the file should be opened.

Syntax

```
file = open("filename", "mode")
```

- **filename** → Name of the file
- **mode** → Specifies how the file will be used (read, write, append, etc.)

- Closing files using close().

In Python, after working with a file (reading or writing), it is important to **close the file using the close() function**. Closing a file releases the system resources that were used while the file was open.

If a file is not closed properly, it may lead to **data loss, memory issues, or the file remaining locked**.

Syntax

```
file.close()
```

4. Reading and Writing Files

- **Reading from a file using read(), readline(), readlines():-**

In Python, files can be read using different methods such as **read()**, **readline()**, and **readlines()**. These methods help in retrieving data from a file in different ways.

1. read() Method

The read() method is used to read the entire content of the file at once.

2. readline() Method

The readline() method is used to read one line at a time from the file.

3. readlines() Method

The readlines() method is used to read all lines of a file and store them in a list.

- **Writing to a file using write() and writelines().:-**

In Python, files can be written using the **write()** and **writelines()** methods. These methods are used to **store data in a file**.

Before writing to a file, it must be opened in **write mode ('w')** or **append mode ('a')**.

1. write() Method

The write() method is used to write a single string into a file

2. writelines() Method

The writelines() method is used to write multiple lines into a file.

5. Exception Handling

- **Introduction to exceptions and how to handle them using try, except, and finally :-**

Python uses the try, except, else, and finally keywords for robust exception handling.

- **try:** This block contains the code that might raise an exception. Python monitors this section for errors during execution.
- **except:** If an exception occurs within the try block, the rest of the try block is skipped, and control is transferred to the matching except block. This block contains the code to handle the specific error (e.g., printing an error message, using a fallback value).
- **else:** This block is optional and executes only if the code in the try block runs without raising any exceptions. It is ideal for code that should only run on a successful execution.
- **finally:** This block always executes, regardless of whether an exception occurred, was caught, or not. It is commonly used for essential cleanup tasks, such as closing files or releasing external resources, to prevent resource leak.

- **Understanding multiple exceptions and custom exceptions:-**

exception handling is used to manage errors that occur during the execution of a program. Sometimes a program may produce **different types of errors**, so Python allows handling **multiple exceptions**. Python also allows programmers to create **custom exceptions** for specific situations.

1. Multiple Exceptions

Multiple exceptions occur when a program may raise more than one type of error. Python allows handling these errors using **multiple except blocks**.

2. Custom Exceptions

A **custom exception** is a user-defined exception created by the programmer to handle specific conditions that are not covered by built-in exceptions.

Custom exceptions are created by **defining a class that inherits from the Exception class** and using the **raise keyword**.

6. Class and Object (OOP Concepts)

• Understanding the concepts of classes, objects, attributes, and methods in Python:-

In Python, Object-Oriented Programming (OOP) uses four core concepts to model real-world entities: classes, objects, attributes, and method.

Classes

A **class** is a blueprint or a template for creating objects. It defines the common properties (attributes) and behaviors (methods) that all objects of that type will have. For example, a Car class defines that all cars have a color and make (attributes) and can start() or stop() (methods)

Objects

An **object** is a specific instance of a class. Each object has its own unique set of data (its state) but shares the structure and methods defined by its class. A red Toyota sedan is a specific object, or instance, of the generic Car class. Objects are created by "instantiating" the class (e.g., my_car = Car("red", "Toyota")).

Attributes

Attributes are variables that store data within a class or object. They represent the state or properties of an object. There are two types:

- **Instance Attributes:** These are unique to each object and are typically defined within the special `__init__` constructor method using the self keyword (e.g., self.name = name). Each object maintains its own copy of instance attributes.
- **Class Attributes:** These are shared by all instances of a class and are defined directly inside the class body, outside of any methods (e.g., species = "canine").

Methods

- **Methods** are functions defined inside a class that describe the behaviors or actions an object can perform. They operate on the data stored in the object's attributes.

- **Instance Methods:** These are the most common and must include `self` as the first parameter, which is a reference to the specific object the method is called on. This allows the method to access and modify that object's instance attributes.
- **Class Methods:** These operate on the class itself and take `cls` as the first parameter (a reference to the class). They are defined using the `@classmethod` decorator and can only access class attributes.
- **Static Methods:** These are general utility functions that have a logical connection to the class but do not need access to instance or class attributes. They use the `@staticmethod` decorator and do not require `self` or `cls` parameters.
- **Difference between local and global variables:-**

Local variable	Global variable
Variables which are declared inside the method and within the scope of class.	Variables which are declared outside the method and within the scope of class.
Local variables are accessible only within the method in which they are declared, they are not accessible outside of that method.	Global variables are accessible everywhere within the scope of a class.
It is mandatory to initialize local variables, if we did not initialize we will get compile time error.	It is not mandatory to initialize, if we did not initialize, <i>NM</i> will take default values based on data types.
There are no types for local variables.	Global variables are divided into 2 types i.e. static and non-static.
Local variables can be default or final.	Global variables can be public, private, protected, default or final.
Local variables are present in Stack area.	Global variables are present in Heap area.

7. Inheritance

- **Single, Multilevel, Multiple, Hierarchical, and Hybrid inheritance in Python:-**

Python supports five main types of inheritance: **Single, Multiple, Multilevel, Hierarchical, and Hybrid**. These types define how classes can build relationships and reuse code from one or multiple parent classes.

Types of Inheritance in Python

Single Inheritance: A child class inherits from only one parent class.

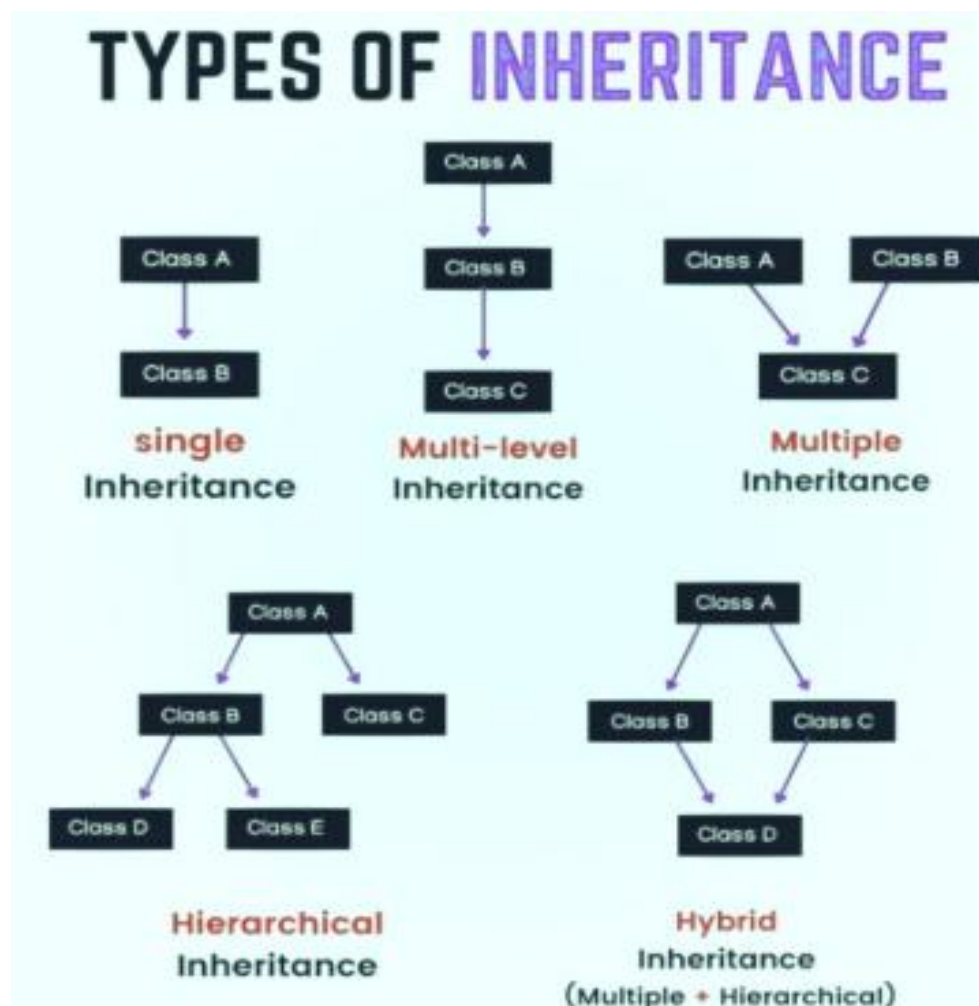
This is the most basic and common form, creating a simple, direct hierarchy.

Multilevel Inheritance: A class inherits from a child class, creating a chain of inheritance (Grandparent -> Parent -> Child).

Multiple Inheritance: A child class inherits from multiple parent classes. This allows the child to combine features from several sources, though it requires careful handling of the Method Resolution Order (MRO) in case of method name conflicts.

Hierarchical Inheritance: Multiple child classes inherit from a single parent class. This is useful for creating several specialized variations of a general class.

Hybrid Inheritance: This is a combination of two or more types of inheritance, such as combining hierarchical and multiple inheritance. It results in a more complex but flexible class structure.



- Using the `super()` function to access properties of the parent class:-

the **super() function** is used to **call methods or access properties of the parent (base) class from the child (derived) class**. It is mainly used in **inheritance** to reuse the functionality of the parent class.

Using super() helps avoid rewriting the same code and makes programs easier to maintain.

Example

```
class Animal:
    def __init__(self, name):
        self.name = name

    def show(self):
        print("Animal name:", self.name)

class Dog(Animal):
    def __init__(self, name):
        super().__init__(name) # Calling parent class constructor

    def sound(self):
        print(self.name, "barks")

d = Dog("Buddy")
d.show()    # Parent class method
d.sound()  # Child class method
```

8. Method Overloading and Overriding

- **Method overloading: defining multiple methods with the same name but different parameters:-**

Method overloading means **defining multiple methods with the same name but different parameters** in a class. The method performs different tasks depending on the **number or type of arguments passed**.

method overloading is directly supported. However, **Python does not support traditional method overloading**, but it can be implemented using **default arguments or variable-length arguments (*args)**.

Default Arguments: You can use default values for parameters, allowing a method to be called with different numbers of arguments.

Variable-Length Arguments (*args and **kwargs): This approach lets a single method accept a dynamic number of positional arguments (*args) or keyword arguments (**kwargs).

Conditional Logic: Inside a single method, you can use conditional statements (like if-elif-else) to check the number or type of arguments passed and adjust the behavior accordingly.

• **Method overriding: redefining a parent class method in the child class:-**

Method overriding is a core concept in object-oriented programming where a subclass provides a specific implementation for a method that is already defined in its superclass [1]. This allows for a child class to modify the behavior of an inherited method, ensuring that when the method is called on an instance of the subclass, the child class's version is executed instead of the parent's [1].

In Python, this is achieved by simply defining a method in the child class with the exact same name and signature (parameters) as the method in the parent class

Key Points:

- **Polymorphism:** Method overriding is an example of polymorphism in Python, where objects of different classes can respond to the same method call in different ways [1].
- **super() function:** You can still call the parent class's method from within the overridden method in the child class using the super() function. This is useful for extending the parent's functionality rather than completely replacing it [1]. For more details, refer to the Python documentation on the super() built-in function.

9. SQLite3 and PyMySQL (Database Connectors)

• **Introduction to SQLite3 and PyMySQL for database connectivity:-**

Python provides several libraries to connect and work with databases. Two commonly used libraries are **SQLite3** and **PyMySQL**. These libraries allow Python programs to **store, retrieve, and manage data in databases**.

1. SQLite3

SQLite3 is a **lightweight, file-based database** that comes **built into Python**. It does not require a separate server process, and the entire database is stored in a **single file**.

Features

- No installation required (included with Python).
- Simple and easy to use.
- Suitable for **small applications and local data storage**.

Example

```
import sqlite3

conn = sqlite3.connect("mydatabase.db")

print("Connected to SQLite database")
```

2. PyMySQL

PyMySQL is a Python library used to connect to a **MySQL database server**. MySQL is a **client-server relational database management system (RDBMS)** used for larger and more scalable applications.

Features

- Requires installation using pip.
- Connects Python programs to a **MySQL server**.
- Suitable for **web applications and large systems**.

Installation: pip install pymysql

Example:-

```
import pymysql
connection = pymysql.connect(
    host="localhost",
    user="root",
    password="1234",
    database="mydatabase"
)

print("Connected to MySQL database" )
```

• Creating and executing SQL queries from Python using these connectors:-

Python can interact with databases like **SQLite** and **MySQL** using database connectors such as **SQLite3** and **PyMySQL**. These connectors allow Python programs to **create tables, insert data, update records, and retrieve data** by executing SQL queries.

1. Using SQLite3

SQLite3 is a **built-in Python module** used to connect to SQLite databases.

2. Using PyMySQL

PyMySQL is used to connect Python programs with a **MySQL database server**.

Using connectors like **SQLite3** and **PyMySQL**, Python can execute **SQL queries** to manage databases. These connectors allow Python programs to **create tables, insert data, update records, and retrieve information from databases** efficiently.

10. Search and Match Function

- **Using re.search() and re.match() functions in Python's re module for pattern matching:-**

the key difference between re.match() and re.search() is the location where they look for a pattern. re.match() checks only at the **beginning of the string**, while re.search() scans the **entire string** for the first occurrence of the pattern.

Both functions return a Match object if a match is found, and None otherwise. It is a best practice to use raw strings (prefixed with r) for regex patterns to avoid issues with backslashes.

re.match()

The re.match() function attempts to match a pattern at the very beginning of the string. It is useful for validating if a string starts with a specific format or prefix.

Syntax: re.match(pattern, string, flags=0)

re.search()

The re.search() function scans through the entire string to find the first location where the pattern matches. It is more flexible than re.match() and typically more used for general keyword or pattern finding within a text.

Syntax: re.search(pattern, string, flags=0)

Feature 	<code>re.match()</code>	<code>re.search()</code>
Search Location	Only at the beginning of the string.	Anywhere in the string.
Behavior	Behaves as if the pattern has <code>^</code> prepended.	Scans the whole string until the first match is found.
Common Use Case	Validating a string's starting format.	Finding a pattern or keyword in a larger body of text.
Return Value	<code>Match</code> object if found at the start, otherwise <code>None</code> .	<code>Match</code> object for the first match found anywhere, otherwise <code>None</code> .

• Difference between search and match:-

Feature	<code>re.match()</code>	<code>re.search()</code>
Search Location	Only at the beginning of the string.	Anywhere in the string.
Return Value	Match object if at the start; otherwise None.	Match object for first match; otherwise None.
Use Case	Match pattern at the start.	Find pattern anywhere in the string.
Example	<code>re.match(r"hello", "hello world")</code> (matches)	<code>re.search(r"world", "hello world")</code> (matches)
Behavior	if No Match Returns None if no match at the start.	Returns None if no match anywhere.
Performance	Faster for start-of-string matching.	More flexible but may be slower.

